ENTREGABLE 7 — INFORME EJECUTIVO Y PRESENTACIÓN FINAL

Proyecto SGA-DO (Sistema de Gestión Académica - DiplomadosOnline)

1. Conclusiones del proyecto

Desarrollar el Sistema de Gestión Académica SGA-DO me permitió entender cómo se construye un backend desde cero. Logré modelar la jerarquía de clases (Persona, Alumno, Profesor, ProgramaAcademico y sus hijas Curso, Diplomado y Bootcamp) aplicando herencia y polimorfismo con el método evaluarAprobacion(alumno), como pidió el profe Rhadamés. También implementé la persistencia en archivos .txt, la pila LIFO para deshacer la última nota y la cola FIFO para generar el reporte de graduandos. El sistema supera los casos de prueba CP-01, CP-02 y CP-03, y resiste las pruebas EVAL-01 (apagón), EVAL-02 (polimorfismo real sin ifs), EVAL-03 (try-catch ante entradas inválidas) y EVAL-04 (gestión de memoria en C++ con new/delete y destructor).

2. Cuadro comparativo: pros y contras de los tres lenguajes

Lenguaje	Pros	Contras
Python	Sintaxis limpia y corta. No necesita puntos y comas. La indentación obliga a ordenar el código. Las listas y collections.deque resuelven pila y cola en una línea. Ideal para prototipar rápido.	No hay encapsulamiento estricto (todo es público por convención). No hay gestión manual de memoria, lo que hace difícil entender qué pasa por debajo. Es más lento en ejecución que Java y C++.
Java	Encapsulamiento real con private y getters/setters. El polimorfismo con @Override y abstract queda muy claro. El manejo de excepciones con try-catch y NumberFormatException es robusto. Buenas colecciones oficiales (Stack, LinkedList).	Es muy verboso: cada clase abre y cierra llaves, hay que declarar todo. Tuve que instalar el JDK 17 y configurar variables de entorno. Los caracteres con tilde se veían raros en la consola de Windows hasta usar chcp 65001.
C++	Control total de la memoria con new y delete. El destructor ~Alumno() se ejecuta solo y evita fugas. Los punteros permiten entender cómo funciona la memoria realmente. Es el más rápido en ejecución de los tres.	Es el más difícil de los tres. La sintaxis de punteros, virtual puro con = 0 y override es compleja. Cualquier descuido con new sin delete genera fugas. Los errores del compilador son largos y confusos. Compilar desde terminal asusta al principio.

3. Experiencia personal con cada lenguaje

Python fue el más amable. Implementé las clases y el polimorfismo casi sin fricciones, y las correcciones del profe (cédula única, no defaultear a Bootcamp, una nota a la vez y cargar los .txt al iniciar) las pude aplicar directo.

Java me costó más. VS Code lanzaba "can't find main method in Persona" hasta que entendí que el main vive dentro de la clase pública que se llama igual que el archivo (SGADO). En la terminal escribí por error javac SGA.DO.java y me decía "file not found". Tuve que instalar el JDK 17 de Eclipse Temurin y usar chcp 65001 para que las tildes se vieran bien. Valoré el orden que impone el encapsulamiento.

C++ fue el más difícil. En el compilador online de Programiz me salió un error de "redefinition of int main" que no entendía hasta darme cuenta de que el compilador traía una plantilla con su propio main y yo había pegado mi código encima sin borrarla. La parte más dura fue la gestión manual de memoria: new para reservar, delete para liberar, y el destructor ~Alumno() que se ejecuta automáticamente. Al cerrar el programa tuve que recorrer las listas con un for y hacer delete de cada alumno y profesor para garantizar cero fugas.

4. Lección final

La conclusión más clara que me deja este proyecto es que más allá de la sintaxis, lo importante es entender la lógica del problema (herencia, polimorfismo, persistencia, estructuras de datos). Una vez clara esa lógica, cambiar de lenguaje es cuestión de adaptar la forma, no el fondo. Python es rápido para prototipar, Java te obliga a ser ordenado y C++ te enseña cómo funciona la memoria realmente. Los tres cumplen el mismo objetivo, cada uno con su propia filosofía.